Radiant College, Manigram Butwal

[ITEX 103] Object Oriented Programming

B Tech Ed. – 2ND SEMESTER

Unit – 4

(Inheritance)

Introduction to Inheritance:

Inheritance allows a class (derived class) to inherit attributes and behaviors from another class (base class). This promotes code reusability and establishes hierarchical relationships between classes.

Base and Derived Classes:

```
// Base class
class Animal {
public:
    void eat() {
        cout << "I can eat!" << endl;
    }
};

// Derived class
class Dog : public Animal {
public:
    void bark() {
        cout << "I can bark! Woof woof!" << endl;
    }
};

int main() {
    Dog myDog;
    myDog.eat();  // Inherited from Animal
    myDog.bark(); // Defined in Dog
    return 0;
}
```

Access Specifiers in Inheritance

Inheritance Type	Public Members	Protected Members	Private Members
public	public	protected	inaccessible
protected	protected	protected	inaccessible
private	private	private	inaccessible

Example:

```
class Base {
public:
    int publicVar;
protected:
    int protectedVar;
private:
    int privateVar;
};

class PublicDerived : public Base {
    // publicVar remains public
    // protectedVar remains protected
    // privateVar is inaccessible
};

class ProtectedDerived : protected Base {
    // publicVar becomes protected
    // protectedVar remains protected
    // privateVar is inaccessible
};
```

```
class PrivateDerived : private Base {
    // publicVar becomes private
    // protectedVar becomes private
    // privateVar is inaccessible
};
```

Derived Class Constructors and Destructors

Derived classes need to initialize both their own members and inherited members.

Constructor Execution Order

1. Base class constructor
2. Member objects' constructors (in declaration order)
3. Derived class constructor

Destructor Execution Order

```
class Base {
public:
    Base() { cout << "Base constructor" << endl; }
    ~Base() { cout << "Base destructor" << endl; }
};

class Derived : public Base {
public:
    Derived() { cout << "Derived constructor" << endl; }
    ~Derived() { cout << "Derived destructor" << endl; }
};

int main() {
    Derived d;
```

```
    return 0;
}

/* Output:
Base constructor
Derived constructor
Derived destructor
Base destructor
*/
```

Single Inheritance

Single inheritance occurs when a derived class inherits from only one base class. This is the simplest and most common form of inheritance, where a class (child) inherits properties and behaviors from a single parent class.

Key Characteristics:

- One base class ➔ One derived class
- Simple hierarchical relationship
- Most straightforward type of inheritance
- Avoids complexity of multiple inheritance
-

Syntax:

```
class DerivedClass : access-specifier BaseClass {
    // Derived class members
};
```

Example:

```
#include <iostream>
#include <string>
using namespace std;

// Base class (Parent)
class Vehicle {
protected:
    string brand;
    int year;

public:
    Vehicle(string b, int y) : brand(b), year(y) {}

    void displayInfo() {
        cout << "Brand: " << brand << "\nYear: " << year << endl;
    }

    void startEngine() {
        cout << "Engine started!" << endl;
    }
};

// Derived class (Child) - Single inheritance
class Car : public Vehicle {
private:
    int numDoors;

public:
    // Derived class constructor
    Car(string b, int y, int doors)
        : Vehicle(b, y),  // Initialize base class
          numDoors(doors) {}  // Initialize derived members

    // Additional functionality specific to Car
    void displayCarDetails() {
        displayInfo();  // Call base class method
```

```
        cout << "Number of doors: " << numDoors << endl;
    }

    void honk() {
        cout << "Honk! Honk!" << endl;
    }
};

int main() {
    // Create Car object
    Car myCar("Toyota", 2023, 4);

    // Access base class functionality
    myCar.startEngine();  // Output: Engine started!

    // Use derived class methods
    myCar.displayCarDetails();
    /* Output:
        Brand: Toyota
        Year: 2023
        Number of doors: 4
    */

    myCar.honk();  // Output: Honk! Honk!

    return 0;
}
```

Multiple Inheritance

Multiple inheritance allows a class to inherit from more than one base class. This enables a derived class to combine features and behaviors from multiple parent classes.

Key Characteristics:

- Derives from two or more base classes
- Combines functionalities from multiple sources
- Requires careful handling to avoid ambiguity
- Can lead to the "diamond problem"

Syntax:

```
class DerivedClass : access-specifier Base1, access-specifier Base2,
... {
    // Derived class members
};
```

Implementation Example:

```
#include <iostream>
#include <string>
using namespace std;

// Base class 1: Car functionality
class Car {
public:
    void drive() {
        cout << "Driving on the road" << endl;
    }
};
```

```cpp
// Base class 2: Airplane functionality
class Airplane {
public:
    void fly() {
        cout << "Flying in the sky" << endl;
    }
};

// Derived class combining both Car and Airplane
class FlyingCar : public Car, public Airplane {
public:
    void transform() {
        cout << "Transforming from car to airplane mode!" << endl;
    }

    void travel() {
        drive();  // Use Car functionality
        transform();
        fly();    // Use Airplane functionality
    }
};

int main() {
    FlyingCar myVehicle;

    cout << "=== Starting Journey ===" << endl;
    myVehicle.travel();

    cout << "\n=== Direct Method Calls ===" << endl;
    myVehicle.drive();  // Call Car method directly
    myVehicle.fly();    // Call Airplane method directly

    return 0;
}
```

Output:

```
=== Starting Journey ===
Driving on the road
Transforming from car to airplane mode!
Flying in the sky

=== Direct Method Calls ===
Driving on the road
Flying in the sky
```

Ambiguity in Multiple Inheritance

When base classes have members with the same name, use the scope resolution operator `::` to specify which base class member you want to access:

```
class Camera {
public:
    void capture() { cout << "Taking photo" << endl; }
};

class Phone {
public:
    void capture() { cout << "Recording audio" << endl; }
};

class Smartphone : public Camera, public Phone {
public:
    void useCamera() {
        Camera::capture();  // Explicitly call Camera's capture
    }
```

```
    void usePhone() {
        Phone::capture();   // Explicitly call Phone's capture
    }
};

int main() {
    Smartphone sp;
    sp.useCamera();  // Output: Taking photo
    sp.usePhone();   // Output: Recording audio

    // sp.capture(); // Error: ambiguous call
    sp.Camera::capture(); // Explicit resolution
    return 0;
}
```

The Diamond Problem and Virtual Inheritance

When two base classes inherit from a common ancestor, it creates ambiguity:

```
class Person {
public:
    string name;
};

class Faculty : public Person {};
class Student : public Person {};

class TeachingAssistant : public Faculty, public Student {
    // This would have two copies of Person!
};
```

Solution: Virtual Inheritance
Ensures only one copy of the common base class is inherited:

```
class Person {
public:
    string name;
};

class Faculty : virtual public Person {};   // Virtual inheritance
class Student : virtual public Person {};   // Virtual inheritance

class TeachingAssistant : public Faculty, public Student {
public:
    TeachingAssistant(string n) {
        name = n;  // No ambiguity - single Person instance
    }

    void display() {
        cout << "Teaching Assistant: " << name << endl;
    }
};

int main() {
    TeachingAssistant ta("Alice Johnson");
    ta.display();  // Output: Teaching Assistant: Alice Johnson
    return 0;
}
```